



INSTITUT  
TEKNOLOGI  
HARAPAN  
BANGSA  
*Veritas vos liberabit*

**MODUL PRAKTIKUM**  
***Teknik Informatika***

**IF-9P2**  
**Pemrograman Piranti Bergerak Lanjut**

Firhat Hidayat, S.Kom., M.T.

2026 © All rights reserved.

Dilarang memperbanyak dan/atau mengcopy sebagian atau seluruh material  
dalam dokumen ini tanpa persetujuan tertulis dari Institut Teknologi Harapan Bangsa

MODUL PRAKTIKUM

# Android Development II

---

## Unit Testing dan UI Testing pada Jetpack Compose

|            |                            |
|------------|----------------------------|
| Case Study | Login Screen Berbasis MVVM |
|------------|----------------------------|

|        |                         |
|--------|-------------------------|
| Durasi | 1 Pertemuan / 150 Menit |
|--------|-------------------------|

|             |                        |
|-------------|------------------------|
| Mata Kuliah | Android Development II |
|-------------|------------------------|

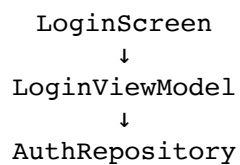
## 1. Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

- Membuat project Android baru berbasis Jetpack Compose.
- Membuat fitur Login Screen menggunakan pola MVVM.
- Memisahkan UI, ViewModel, State, dan Repository.
- Membuat Unit Test untuk logika login pada ViewModel.
- Membuat UI Test untuk interaksi Login Screen.
- Menjalankan test melalui Android Studio.

## 2. Gambaran Arsitektur

Pada praktikum ini, aplikasi dibuat dengan struktur sederhana:



Penjelasan:

| Komponen           | Tanggung Jawab                              |
|--------------------|---------------------------------------------|
| LoginScreen        | Menampilkan UI dan menerima input user      |
| LoginViewModel     | Menyimpan state dan menjalankan logic login |
| AuthRepository     | Menyediakan proses autentikasi              |
| FakeAuthRepository | Repository palsu untuk testing              |

Testing akan dilakukan pada dua level:

| Jenis Test | Target         |
|------------|----------------|
| Unit Test  | LoginViewModel |
| UI Test    | LoginScreen    |

### 3. Persiapan Project Baru

#### Langkah 1 - Buat Project

- Buka Android Studio.
- Pilih New Project.
- Pilih template: Empty Activity.

Isi konfigurasi:

Name: LoginTestingApp  
Package name: com.example.logintestingapp  
Language: Kotlin  
Minimum SDK: API 24

- Klik Finish.
- Tunggu Gradle Sync selesai.

### 4. Dependency Testing

Buka file: app/build.gradle.kts

Pastikan dependency testing tersedia. Tambahkan atau sesuaikan bagian berikut:

```
dependencies {  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")  
    testImplementation("junit:junit:4.13.2")  
    androidTestImplementation("androidx.test.ext:junit:1.2.1")  
    androidTestImplementation("androidx.test.espresso:espresso-core:3.6.1")  
    androidTestImplementation("androidx.compose.ui:ui-test-junit4")  
    debugImplementation("androidx.compose.ui:ui-test-manifest")  
}
```

*Catatan: Versi library di atas perlu dicek ulang terhadap rilis terbaru (Maven/Google Maven) sebelum digunakan - versi dependency Android sering berubah. Jika project baru sudah menggunakan Compose BOM, dependency Compose UI Test tidak perlu menulis versi secara manual.*

Compose UI testing menyediakan API untuk menemukan node UI, melakukan aksi pengguna, dan membuat assertion terhadap tampilan Compose (semantics, finder, action, assertion).

## 5. Struktur Folder

Buat struktur folder berikut:

```
app/src/main/java/com/example/logintestingapp
├── data
│   └── AuthRepository.kt
├── ui
│   └── login
│       ├── LoginScreen.kt
│       ├── LoginUiState.kt
│       └── LoginViewModel.kt
└── MainActivity.kt
```

Folder testing:

```
app/src/test/java/com/example/logintestingapp
└── ui/login/LoginViewModelTest.kt
```

```
app/src/androidTest/java/com/example/logintestingapp
└── ui/login/LoginScreenTest.kt
```

*Catatan: Local unit test ditulis di src/test/ dan berjalan di JVM lokal; UI/instrumented test ditulis di src/androidTest/ dan berjalan pada emulator atau device.*

## 6. Membuat AuthRepository

Buat file: data/AuthRepository.kt

```
package com.example.logintestingapp.data

interface AuthRepository {
    fun login(username: String, password: String): Boolean
}

class FakeAuthRepository : AuthRepository {
    override fun login(username: String, password: String): Boolean {
        return username == "admin" && password == "123456"
    }
}
```

Penjelasan:

- AuthRepository adalah kontrak autentikasi.
- FakeAuthRepository digunakan sebagai implementasi sederhana.
- Pada aplikasi nyata, repository dapat memanggil API.
- Pada praktikum ini, data dibuat statis agar mudah dites.

## 7. Membuat LoginUiState

Buat file:

**ui/login/LoginUiState.kt**

```
package com.example.logintestingapp.ui.login

data class LoginUiState(
    val username: String = "",
    val password: String = "",
    val isSuccess: Boolean = false,
    val errorMessage: String? = null
)
```

**LoginUiState** menyimpan seluruh state yang dibutuhkan oleh Login Screen:

| State          | Fungsi                       |
|----------------|------------------------------|
| username       | Input username               |
| password       | Input password               |
| isLoginSuccess | Status login berhasil        |
| errorMessage   | Pesan error saat login gagal |

*Catatan: Jetpack Compose mendorong developer untuk eksplisit dalam menyimpan dan menggunakan state, karena UI akan diperbarui ketika state yang dibaca oleh composable berubah.*

## 8. Membuat LoginViewModel

Buat file:

**ui/login/LoginViewModel.kt**

```
package com.example.logintestingapp.ui.login

import androidx.lifecycle.ViewModel
import com.example.logintestingapp.data.AuthRepository

class LoginViewModel(
    private val authRepository: AuthRepository
) : ViewModel() {

    var uiState = LoginUiState()
    private set

    fun onUsernameChange(value: String) {
        uiState = uiState.copy(username = value, errorMessage = null)
    }
}
```

```

fun onPasswordChange(value: String) {
    uiState = uiState.copy(password = value, errorMessage = null)
}

fun login() {
    if (uiState.username.isBlank()) {
        uiState = uiState.copy(
            isLoggedIn = false,
            errorMessage = "Username tidak boleh kosong"
        )
        return
    }
    if (uiState.password.isBlank()) {
        uiState = uiState.copy(
            isLoggedIn = false,
            errorMessage = "Password tidak boleh kosong"
        )
        return
    }
    val result = authRepository.login(
        username = uiState.username,
        password = uiState.password
    )
    uiState = if (result) {
        uiState.copy(isLoggedIn = true, errorMessage = null)
    } else {
        uiState.copy(
            isLoggedIn = false,
            errorMessage = "Username atau password salah"
        )
    }
}
}

```

*Catatan: Kode di atas sengaja dibuat sederhana untuk kepentingan awal testing. Pada project Android yang lebih serius, uiState biasanya menggunakan mutableStateOf() atau StateFlow. Untuk praktikum awal testing, struktur sederhana ini cukup untuk memahami Unit Test.*

## 9. Membuat LoginScreen

Buat file:

**ui/login/LoginScreen.kt**

```

package com.example.logintestingapp.ui.login

import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.material3.TextField
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.testTag
import androidx.compose.ui.unit.dp

@Composable

```

```

fun LoginScreen(
    uiState: LoginUiState,
    onUsernameChange: (String) -> Unit,
    onPasswordChange: (String) -> Unit,
    onLoginClick: () -> Unit
) {
    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text(text = "Login Screen")
        Spacer(modifier = Modifier.height(16.dp))
        TextField(
            value = uiState.username,
            onValueChange = onUsernameChange,
            label = { Text("Username") },
            modifier = Modifier.fillMaxWidth().testTag("usernameField")
        )
        Spacer(modifier = Modifier.height(8.dp))
        TextField(
            value = uiState.password,
            onValueChange = onPasswordChange,
            label = { Text("Password") },
            modifier = Modifier.fillMaxWidth().testTag("passwordField")
        )
        Spacer(modifier = Modifier.height(16.dp))
        Button(
            onClick = onLoginClick,
            modifier = Modifier.fillMaxWidth().testTag("loginButton")
        ) {
            Text("Login")
        }
        Spacer(modifier = Modifier.height(16.dp))
        if (uiState.isLoginSuccess) {
            Text(text = "Login berhasil", modifier =
Modifier.testTag("successMessage"))
        }
        if (uiState.errorMessage != null) {
            Text(text = uiState.errorMessage, modifier =
Modifier.testTag("errorMessage"))
        }
    }
}

```

*Catatan: testTag() digunakan agar UI Test lebih stabil. Tanpa testTag, test biasanya mencari komponen berdasarkan teks - masih bisa, tetapi lebih mudah rusak jika teks UI berubah.*



## 10. Menghubungkan ke MainActivity

Buka MainActivity.kt, ganti isinya menjadi:

```
package com.example.logintestingapp

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import com.example.logintestingapp.data.FakeAuthRepository
import com.example.logintestingapp.ui.login.LoginScreen
import com.example.logintestingapp.ui.login.LoginViewModel
import com.example.logintestingapp.ui.theme.LoginTestingAppTheme

class MainActivity : ComponentActivity() {

    private val viewModel = LoginViewModel(authRepository =
FakeAuthRepository())
    private var uiState by mutableStateOf(viewModel.uiState)

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            LoginTestingAppTheme {
                LoginScreen(
                    uiState = uiState,
                    onUsernameChange = {
                        viewModel.onUsernameChange(it)
                        uiState = viewModel.uiState
                    },
                    onPasswordChange = {
                        viewModel.onPasswordChange(it)
                        uiState = viewModel.uiState
                    },
                    onLoginClick = {
                        viewModel.login()
                        uiState = viewModel.uiState
                    }
                )
            }
        }
    }
}
```

Jalankan aplikasi. Test manual:

| Input           | Output                       |
|-----------------|------------------------------|
| username kosong | Username tidak boleh kosong  |
| password kosong | Password tidak boleh kosong  |
| admin / 123456  | Login berhasil               |
| user / 111111   | Username atau password salah |

## 11. Membuat Unit Test LoginViewModel

Buka folder:

**app/src/test/java/com/example/logintestingapp/ui/login**

Buat file:

**LoginViewModelTest.kt**

```
package com.example.logintestingapp.ui.login

import com.example.logintestingapp.data.AuthRepository
import org.junit.Assert.*
import org.junit.Test

class LoginViewModelTest {

    private class SuccessAuthRepository : AuthRepository {
        override fun login(username: String, password: String) = true
    }

    private class FailedAuthRepository : AuthRepository {
        override fun login(username: String, password: String) = false
    }

    @Test
    fun usernameChange_shouldUpdateUsernameState() {
        val viewModel = LoginViewModel(authRepository =
SuccessAuthRepository())
        viewModel.onUsernameChange("admin")
        assertEquals("admin", viewModel.uiState.username)
    }

    @Test
    fun passwordChange_shouldUpdatePasswordState() {
        val viewModel = LoginViewModel(authRepository =
SuccessAuthRepository())
        viewModel.onPasswordChange("123456")
        assertEquals("123456", viewModel.uiState.password)
    }
}
```

```

@Test
fun login_withEmptyUsername_shouldShowUsernameError() {
    val viewModel = LoginViewModel(authRepository =
SuccessAuthRepository())
    viewModel.onPasswordChange("123456")
    viewModel.login()
    assertFalse(viewModel.uiState.isLoginSuccess)
    assertEquals("Username tidak boleh kosong",
viewModel.uiState.errorMessage)
}

@Test
fun login_withEmptyPassword_shouldShowPasswordError() {
    val viewModel = LoginViewModel(authRepository =
SuccessAuthRepository())
    viewModel.onUsernameChange("admin")
    viewModel.login()
    assertFalse(viewModel.uiState.isLoginSuccess)
    assertEquals("Password tidak boleh kosong",
viewModel.uiState.errorMessage)
}

@Test
fun login_withValidCredential_shouldShowSuccess() {
    val viewModel = LoginViewModel(authRepository =
SuccessAuthRepository())
    viewModel.onUsernameChange("admin")
    viewModel.onPasswordChange("123456")
    viewModel.login()
    assertTrue(viewModel.uiState.isLoginSuccess)
    assertNull(viewModel.uiState.errorMessage)
}

@Test
fun login_withInvalidCredential_shouldShowError() {
    val viewModel = LoginViewModel(authRepository =
FailedAuthRepository())
    viewModel.onUsernameChange("admin")
    viewModel.onPasswordChange("wrongpassword")
    viewModel.login()
    assertFalse(viewModel.uiState.isLoginSuccess)
    assertEquals("Username atau password salah",
viewModel.uiState.errorMessage)
}
}

```

## 12. Menjalankan Unit Test

### Cara 1 - Dari Android Studio

- Buka file LoginViewModelTest.kt.
- Klik icon run hijau di sebelah nama class.
- Pilih Run 'LoginViewModelTest'.

Jika berhasil, semua test akan berwarna hijau.

### Cara 2 - Dari Terminal

```
./gradlew test
```

```
# Windows:  
gradlew test
```

Output yang diharapkan: BUILD SUCCESSFUL

## 13. Analisis Unit Test

| Test                                            | Tujuan                      |
|-------------------------------------------------|-----------------------------|
| usernameChange_shouldUpdateUsernameState        | Memastikan username berubah |
| passwordChange_shouldUpdatePasswordState        | Memastikan password berubah |
| login_withEmptyUsername_shouldShowUsernameError | Validasi username kosong    |
| login_withEmptyPassword_shouldShowPasswordError | Validasi password kosong    |
| login_withValidCredential_shouldShowSuccess     | Login berhasil              |
| login_withInvalidCredential_shouldShowError     | Login gagal                 |

Poin penting:

- Unit Test tidak membuka aplikasi.
- Unit Test tidak membutuhkan emulator.
- Unit Test fokus pada logic.

## 14. Membuat UI Test LoginScreen

Buka folder:

**app/src/androidTest/java/com/example/logintestingapp/ui/login**

Buat file:

**LoginScreenTest.kt**

```

package com.example.logintestingapp.ui.login

import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.test.*
import androidx.compose.ui.test.junit4.createComposeRule
import androidx.test.ext.junit.runners.AndroidJUnit4
import com.example.logintestingapp.ui.theme.LoginTestingAppTheme
import org.junit.Rule
import org.junit.Test
import org.junit.runner.RunWith

@RunWith(AndroidJUnit4::class)
class LoginScreenTest {

    @get:Rule
    val composeTestRule = createComposeRule()

    @Test
    fun loginScreen_shouldDisplayAllComponents() {
        composeTestRule.setContent {
            LoginTestingAppTheme {
                LoginScreen(
                    uiState = LoginUiState(),
                    onUsernameChange = {},
                    onPasswordChange = {},
                    onLoginClick = {}
                )
            }
        }
        composeTestRule.onNodeWithText("Login Screen").assertExists()
        composeTestRule.onNodeWithTag("usernameField").assertExists()
        composeTestRule.onNodeWithTag("passwordField").assertExists()
        composeTestRule.onNodeWithTag("loginButton").assertExists()
    }

    @Test
    fun loginScreen_whenSuccess_shouldDisplaySuccessMessage() {
        composeTestRule.setContent {
            LoginTestingAppTheme {
                LoginScreen(
                    uiState = LoginUiState(
                        username = "admin",
                        password = "123456",
                        isLoginSuccess = true
                    ),
                    onUsernameChange = {},
                    onPasswordChange = {},
                    onLoginClick = {}
                )
            }
        }
        composeTestRule.onNodeWithTag("successMessage").assertExists()
        composeTestRule.onNodeWithText("Login berhasil").assertExists()
    }
}

```

```

@Test
fun loginScreen_whenError_shouldDisplayErrorMessage() {
    composeTestRule.setContent {
        LoginTestingAppTheme {
            LoginScreen(
                uiState = LoginUiState(
                    errorMessage = "Username atau password salah"
                ),
                onUsernameChange = {},
                onPasswordChange = {},
                onLoginClick = {}
            )
        }
    }
    composeTestRule.onNodeWithTag("errorMessage").assertExists()
    composeTestRule.onNodeWithText("Username atau password
salah").assertExists()
}

// Diperbaiki: memakai state hoisting (remember/mutableStateOf) agar
// onUsernameChange benar-benar memperbarui value yang ditampilkan
TextField.
// Versi asli memakai onUsernameChange = {} sehingga TextField tidak
pernah
// ter-update dan assertion di bawah akan gagal.
@Test
fun usernameField_shouldAcceptInput() {
    composeTestRule.setContent {
        var uiState by remember { mutableStateOf(LoginUiState()) }
        LoginTestingAppTheme {
            LoginScreen(
                uiState = uiState,
                onUsernameChange = { uiState = uiState.copy(username =
it) },
                onPasswordChange = { uiState = uiState.copy(password =
it) },
                onLoginClick = {}
            )
        }
    }

    composeTestRule.onNodeWithTag("usernameField").performTextInput("admin")
    composeTestRule.onNodeWithText("admin").assertExists()
}

@Test
fun loginButton_shouldBeClickable() {
    var isClicked = false
    composeTestRule.setContent {
        LoginTestingAppTheme {
            LoginScreen(
                uiState = LoginUiState(),
                onUsernameChange = {},
                onPasswordChange = {},
                onLoginClick = { isClicked = true }
            )
        }
    }
}

```

```

        composeTestRule.onNodeWithTag("loginButton").performClick()
        assert(isClicked)
    }
}

```

*Catatan: createComposeRule() menguji composable secara langsung; setContent {} menampilkan composable yang ingin dites; onNodeWithTag() mencari komponen berdasarkan testTag; performClick() mensimulasikan klik user; performTextInput() mensimulasikan input teks; assertExists() memastikan komponen muncul di layar.*

## 15. Menjalankan UI Test

UI Test harus dijalankan pada emulator atau device fisik.

- Jalankan emulator atau hubungkan device.
- Buka file LoginScreenTest.kt.
- Klik icon run hijau di sebelah nama class.
- Pilih Run 'LoginScreenTest'.

Atau melalui terminal:

```

./gradlew connectedAndroidTest

# Windows:
gradlew connectedAndroidTest

```

Jika berhasil: BUILD SUCCESSFUL

## 16. Mengapa State Hoisting Penting pada UI Test

Pada **LoginScreen**, parameter value **TextField** sepenuhnya dikontrol oleh **uiState** (controlled component). Jika **onUsernameChange** tidak memperbarui **uiState** yang dibaca ulang oleh composable (misalnya dibiarkan kosong, {}), maka **TextField** akan tetap menampilkan nilai lama setelah recomposition, input yang diketik tidak benar-benar tersimpan di layar.

Karena itu, test yang ingin memverifikasi bahwa input benar-benar muncul di layar wajib menggunakan state yang bisa berubah, misalnya dengan remember { mutableStateOf(...) } di dalam **setContent**, seperti pada test **usernameField\_shouldAcceptInput** di atas.

## 17. UI Test dengan Full Flow (Sukses dan Gagal)

Tambahkan test berikut untuk mensimulasikan alur login yang lebih nyata, dari input sampai hasil ditampilkan:

```
@Test
fun loginScreen_fullFlowSuccess_shouldDisplaySuccessMessage() {
    composeTestRule.setContent {
        var uiState by remember { mutableStateOf(LoginUiState()) }
        LoginTestingAppTheme {
            LoginScreen(
                uiState = uiState,
                onUsernameChange = { uiState = uiState.copy(username =
it) },
                onPasswordChange = { uiState = uiState.copy(password =
it) },
                onLoginClick = {
                    uiState = if (uiState.username == "admin" &&
uiState.password == "123456") {
                        uiState.copy(isLoginSuccess = true, errorMessage =
null)
                    } else {
                        uiState.copy(
                            isLoginSuccess = false,
                            errorMessage = "Username atau password salah"
                        )
                    }
                }
            )
        }
    }

    composeTestRule.onNodeWithTag("usernameField").performTextInput("admin")

    composeTestRule.onNodeWithTag("passwordField").performTextInput("123456")
    composeTestRule.onNodeWithTag("loginButton").performClick()
    composeTestRule.onNodeWithText("Login berhasil").assertExists()
}

@Test
fun loginScreen_fullFlowFailed_shouldDisplayErrorMessage() {
    composeTestRule.setContent {
        var uiState by remember { mutableStateOf(LoginUiState()) }
        LoginTestingAppTheme {
            LoginScreen(
                uiState = uiState,
                onUsernameChange = { uiState = uiState.copy(username =
it) },
                onPasswordChange = { uiState = uiState.copy(password =
it) },
                onLoginClick = {
                    uiState = if (uiState.username == "admin" &&
uiState.password == "123456") {
                        uiState.copy(isLoginSuccess = true, errorMessage =
null)
                    } else {
                        uiState.copy(
```



```

        isLoginSuccess = false,
        errorMessage = "Username atau password salah"
    )
}
}
}

composeTestRule.onNodeWithTag("usernameField").performTextInput("admin")

composeTestRule.onNodeWithTag("passwordField").performTextInput("wrong")
    composeTestRule.onNodeWithTag("loginButton").performClick()
    composeTestRule.onNodeWithText("Username atau password
salah").assertExists()
}

```

*Catatan: Gunakan pola remember { mutableStateOf(...) } ini (bukan var uiState biasa di luar Composable) agar Compose benar-benar melakukan recomposition setiap kali state berubah.*

## 18. Interpretasi Hasil Testing

Jika semua test berhasil (Tests passed), artinya:

- ViewModel berhasil memproses input.
- Validasi login berjalan.
- UI menampilkan komponen yang benar.
- UI menampilkan pesan sukses dan error.
- Tombol dan TextField dapat berinteraksi.

Jika test gagal, periksa:

- Apakah teks sama persis?
- Apakah testTag benar?
- Apakah import sudah benar?
- Apakah emulator berjalan?
- Apakah dependency testing sudah tersedia?

## 19. Kesalahan Umum

### 1. Salah Folder Test

Unit Test harus di **src/test**. UI Test harus di **src/androidTest**.

### 2. Lupa Menambahkan testTag

Jika test menggunakan onNodeWithTag("usernameField"), maka composable harus memiliki Modifier.testTag("usernameField").

### 3. UI Test Dijalankan Tanpa Emulator

UI Test butuh emulator atau device. Unit Test tidak butuh emulator.

## 4. State Tidak Berubah Saat UI Test

Masalah: `var uiState = LoginUiState()` di luar Composable — perubahan biasa tidak selalu memicu recomposition.

Solusi:

```
var uiState by remember {  
    mutableStateOf(LoginUiState())  
}
```

## 20. Tugas Praktikum

Kembangkan Login App di atas dengan aturan berikut.

### Unit Test (minimal 6)

- Username berubah ketika input username diberikan.
- Password berubah ketika input password diberikan.
- Login gagal jika username kosong.
- Login gagal jika password kosong.
- Login berhasil jika username dan password benar.
- Login gagal jika username dan password salah.

### UI Test (minimal 4)

- Semua komponen Login Screen tampil.
- TextField username dapat menerima input.
- Login sukses menampilkan pesan Login berhasil.
- Login gagal menampilkan pesan error.

## 21. Deliverable

Mahasiswa mengumpulkan:

- Link GitHub repository.
- Screenshot hasil Unit Test.
- Screenshot hasil UI Test.

Penjelasan singkat:

- Apa yang diuji oleh Unit Test?
- Apa yang diuji oleh UI Test?
- Mengapa ViewModel lebih cocok diuji dengan Unit Test?